32

常见并发问题

多年来，研究人员花了大量时间和精力研究并发 bug（concurrency bug）。早期的大部分工作都集中在**死锁**（deadlock）上，前几章已经提到过这个话题，本章会深入讨论它 [C+71]。较新的工作则着重研究其他类型的常见并发 bug，即非死锁 bug。本章简要介绍在真实代码库中发现的几个并发问题，好让我们更清楚该提防什么。这就引出了本章的核心问题：

关键问题：如何处理常见的并发 bug
并发 bug 往往呈现为几种常见的模式。知道该提防哪些模式，是写出更健壮、更正确的并发代码（concurrent code）的第一步。

32.1 有哪些类型的 bug?

第一个、也是最明显的问题是：复杂的并发程序（concurrent program）中会出现哪些类型的并发 bug? 这个问题一般很难回答，好在已经有人替我们做了这项工作。具体来说，我们依据的是 Lu 等人的一项研究 [L+08]，他们非常细致地分析了一批流行的并发应用程序，弄清了实践中会出现哪些类型的 bug。

这项研究着眼于四个重要的大型开源应用程序：MySQL（流行的数据库管理系统）、Apache（知名的 Web 服务器）、Mozilla（著名的 Web 浏览器）和 OpenOffice（MS Office 套件的免费版本，确实有人在用）。作者考察了这些代码库中已经发现并修复的并发 bug，从开发者的这些工作中整理出了一份定量的 bug 分析。这些结果能帮你看清成熟的代码库中实际会出现哪些问题。

应用程序	用途	非死锁	死锁
MySQL	数据库服务器	14	9
Apache	Web 服务器	13	4
Mozilla	Web 浏览器	41	16
OpenOffice	办公套件	6	2
总计		74	31

图 32.1：**现代应用程序中的 bug**

图 32.1 汇总了 Lu 和同事研究过的 bug。可以看到，bug 一共有 105 个，其中大多数（74 个）是非死锁 bug，其余 31 个是死锁 bug。此外，还可以看到每个应用程序各研究了多少个 bug：OpenOffice 总共只有 8 个并发 bug，而 Mozilla 有将近 60 个。

接下来，我们更深入地讨论非死锁和死锁这两类 bug。第一类是非死锁 bug，我们借助这项研究中的例子来展开讨论；第二类是死锁 bug，我们介绍人们为预防、避免或处理死锁做过的大量工作。

32.2 非死锁 bug

根据 Lu 的研究，非死锁 bug 占了并发 bug 的大多数。但这些 bug 都是什么类型？它们是怎么产生的？又该怎么修复？下面讨论 Lu 等人发现的两大类非死锁 bug：**原子性违反**（atomicity violation）bug 和**顺序违反**（order violation）bug。

原子性违反 bug

我们遇到的第一类问题叫作**原子性违反**（atomicity violation）。下面是在 MySQL 中发现的一个简单例子。在看解释之前，先试着找出 bug 在哪里。动手试试！

```
1 Thread 1::
2 if (thd->proc_info) {
3   fputs(thd->proc_info, ...);
4 }
5
6 Thread 2::
7 thd->proc_info = NULL;
```

图 32.2：**原子性违反（`atomicity.c`）**

在这个例子中，两个线程（thread）都会访问结构体 `thd` 中的字段 `proc_info`。第一个线程检查这个值是否非 NULL，然后打印它；第二个线程把它设为 NULL。显然，如果第一个线程做完检查、但在调用 `fputs` 之前被中断（interrupted），第二个线程就可能在这中间运行，把指针设为 NULL；等第一个线程恢复运行时，`fputs` 会解引用一个 NULL 指针，程序随即崩溃。

按照 Lu 等人的说法，原子性违反（atomicity violation）更正式的定义是："多次内存访问之间期望的可串行性（serializability）被破坏了，即某段代码本应是原子的（atomic），但执行时并没有保证原子性。"在上面的例子中，代码对"检查 `proc_info` 是否非 NULL"和"在 `fputs()` 调用中使用 `proc_info`"这两步有一个*原子性假设*（atomicity assumption，这是 Lu 的说法）；一旦这个假设不成立，代码就无法按预期工作。

这类问题的修复方法通常（但并不总是）很简单。你能想到怎么修复上面的代码吗？

在图 32.3 的解决方案中，我们只是在引用共享变量的地方前后加锁（locking），确保两个线程访问 `proc_info` 字段时都持有锁（`proc_info_lock`）。当然，其他访问这个结构体的代码，也应该先获取这把锁（lock）再访问。

```
1  pthread_mutex_t proc_info_lock =
2    PTHREAD_MUTEX_INITIALIZER;
3
4  Thread 1::
5  pthread_mutex_lock(&proc_info_lock);
6  if (thd->proc_info) {
7    fputs(thd->proc_info, ...);
8  }
9  pthread_mutex_unlock(&proc_info_lock);
10
11 Thread 2::
12 pthread_mutex_lock(&proc_info_lock);
13 thd->proc_info = NULL;
14 pthread_mutex_unlock(&proc_info_lock);
```

图 32.3：**修复后的原子性违反（`atomicity_fixed.c`）**

顺序违反 bug

Lu 等人发现的另一类常见非死锁 bug 叫作**顺序违反**（order violation）。再看一个简单的例子。同样，看看你能不能找出下面的代码为什么有 bug。

```
1  Thread 1::
2  void init() {
3    mThread = PR_CreateThread(mMain, ...);
4  }
5
6  Thread 2::
7  void mMain(...) {
8    mState = mThread->State;
9  }
```

图 32.4：**顺序 bug（`ordering.c`）**

你可能已经发现了，线程 2 的代码似乎假定变量 `mThread` 已经初始化（而且不是 NULL）；然而，如果线程 2 一创建就立即运行，那么它在 `mMain()` 中访问 `mThread` 时，`mThread` 的值还没有设置好，程序很可能因为解引用 NULL 指针而崩溃。注意，我们假设 `mThread` 的初值是 NULL；如果不是，情况可能会更离奇，因为线程 2 会通过这次解引用访问任意的内存位置。

顺序违反（order violation）更正式的定义是：“两次（或两组）内存访问之间期望的顺序被颠倒了（即 A 应该总在 B 之前执行，但执行时并没有保证这个顺序）”[L+08]。

修复这类 bug 的办法一般是强制规定顺序。前面讨论过，使用**条件变量**（condition variable）可以简单而稳妥地在现代代码库中加入这种同步。因此，上面的例子可以改写成图 32.5 那样。

在修复后的代码中，我们加了一个条件变量 `mtCond` 和对应的锁 `mtLock`，还有一个状态变量（state variable）`mtInit`。初始化代码运行时，会把 `mtInit` 的状态设为 1，并发

```
1  pthread_mutex_t mtLock = PTHREAD_MUTEX_INITIALIZER;
2  pthread_cond_t  mtCond = PTHREAD_COND_INITIALIZER;
3  int mtInit             = 0;
4
5  Thread 1::
6  void init() {
7      ...
8      mThread = PR_CreateThread(mMain, ...);
9
10     // 发信号，表示线程已经创建好了……
11     pthread_mutex_lock(&mtLock);
12     mtInit = 1;
13     pthread_cond_signal(&mtCond);
14     pthread_mutex_unlock(&mtLock);
15     ...
16 }
17
18 Thread 2::
19 void mMain(...) {
20     ...
21     // 等待线程初始化完成……
22     pthread_mutex_lock(&mtLock);
23     while (mtInit == 0)
24         pthread_cond_wait(&mtCond, &mtLock);
25     pthread_mutex_unlock(&mtLock);
26
27     mState = mThread->State;
28     ...
29 }
```

图 32.5：**修复顺序违反（`ordering_fixed.c`）**

出信号（signal），表示已经完成。如果线程 2 在此之前就已运行，它会等待这个信号和相应的状态变化；如果它在此之后才运行，就会检查状态，发现初始化已经完成（即 `mtInit` 已设为 1），于是照常继续执行。注意，我们其实也可以直接把 `mThread` 当作状态变量（state variable），但为了简单起见，这里没有这样做。线程之间的顺序很重要时，条件变量或信号量（semaphore）就能派上用场。

非死锁 bug：小结

Lu 等人研究的非死锁 bug 中，很大一部分（97%）不是原子性违反（atomicity violation）就是顺序违反（order violation）。因此，只要仔细思考这两种 bug 模式，程序员很可能就能更好地避免它们。此外，今后出现的自动代码检查工具，大概也应该重点关注这两类 bug，因为在已部署的软件中发现的非死锁 bug，绝大部分都属于这两类。

遗憾的是，并不是所有 bug 都像上面的例子这样容易修复。要修好有些 bug，得更深入地理解程序在做什么，或者大规模重组代码或数据结构（data structure）。更多细节请阅读 Lu 等人那篇出色的（而且好读的）论文。

 © 2008–26, ARPACI-DUSSEAU

32.3 死锁 bug

除了上面提到的并发 bug，在许多加锁协议（locking protocol）复杂的并发系统中，还会出现一个经典问题，叫作**死锁**（deadlock）。例如，线程 1 持有锁 L1，同时在等另一把锁 L2；不巧的是，持有 L2 的线程 2 又在等 L1 被释放。这时就会发生死锁。下面这段代码演示了这种潜在的死锁：

```
Thread 1:                       Thread 2:
pthread_mutex_lock(L1);         pthread_mutex_lock(L2);
pthread_mutex_lock(L2);         pthread_mutex_lock(L1);
```

图 32.6：**简单的死锁（`deadlock.c`）**

注意，这段代码运行时并不一定会发生死锁，只是有可能发生。例如，线程 1 拿到锁 L1 后，发生了一次上下文切换（context switch），换成线程 2 运行。这时线程 2 拿到 L2，又试图获取 L1。于是就发生了死锁，因为两个线程都在等对方，谁都无法运行。图 32.7 用图形描述了这种情况；图中出现了**环**（cycle），就表明发生了死锁。

这张图应该把问题说清楚了。那么，程序员写代码时该怎样应对死锁呢？

关键问题：如何应对死锁
我们应该如何构建系统，来预防、避免死锁，或者至少能检测出死锁并从中恢复？在今天的系统中，死锁是一个真实存在的问题吗？

为什么会发生死锁？

你可能会觉得，上面那种简单的死锁很容易避免。例如，只要线程 1 和线程 2 都按相同的顺序获取锁，就永远不会发生死锁。那么，死锁为什么还会发生呢？

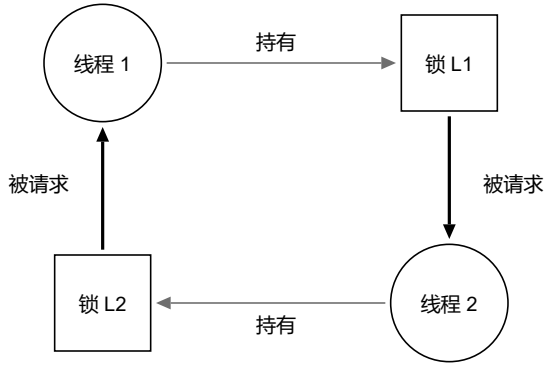


图 32.7：**死锁依赖图**

一个原因是，在大型代码库中，组件之间会形成复杂的依赖关系。以操作系统为例：虚拟内存（virtual memory）系统可能需要访问文件系统（file system），才能从磁盘调入一个块（block）；而文件系统随后可能又需要一页内存来读入这个块，于是又要找虚拟内存系统。因此，在大型系统中设计加锁策略（locking strategy）时必须非常小心，以免代码中自然形成的循环依赖导致死锁。

另一个原因在于**封装**（encapsulation）的本质。作为软件开发者，我们学的是隐藏实现细节，让软件更容易做到模块化（modularity）。遗憾的是，这种模块化和加锁配合得并不好。正如 Jula 等人指出的 [J+08]，有些看似无害的接口简直是在引诱你陷入死锁。以 Java 的 Vector 类和它的方法 `AddAll()` 为例，这个例程的调用方式如下：

```
Vector v1, v2;
v1.AddAll(v2);
```

由于这个方法需要做到多线程安全（multi-thread safe），所以在内部，它既要获取接收添加内容的那个向量（`v1`）的锁，也要获取参数（`v2`）的锁。为了把 `v2` 的内容添加到 `v1`，这个例程按某种任意定下的顺序（比如先 `v1` 后 `v2`）获取这两把锁。如果另一个线程几乎在同一时刻调用了 `v2.AddAll(v1)`，就有可能发生死锁，而调用它的应用程序对此几乎毫无察觉。

产生死锁的条件

要发生死锁，必须同时满足四个条件 [C+71]：

- **互斥**（mutual exclusion）：线程独占它们所需的资源（例如，一个线程拿到了一把锁）。
- **持有并等待**（hold-and-wait）：线程持有已分配给它们的资源（例如已经获得的锁），同时还在等待其他资源（例如它们想要获取的锁）。
- **不可抢占**（no preemption）：不能把资源（例如锁）从持有它的线程那里强行夺走。
- **循环等待**（circular wait）：存在一条由线程组成的环形链，链中的每个线程都持有一个或多个资源（例如锁），而链中的下一个线程正在请求这些资源。

这四个条件中只要有一个不满足，就不会发生死锁。因此，我们先来探讨预防死锁的技术；每种策略都设法阻止上面某一个条件成立，各自都是处理死锁问题的一种办法。

预防

循环等待

最实用的预防技术（当然也是经常采用的一种）大概是：编写加锁代码时，确保永远不会引起循环等待（circular wait）。最简单的做法是为获取锁规定一个**全序**（total ordering）。例如，如果系统中只有两把锁（L1 和 L2），只要始终先获取 L1、再获取 L2，就能预防死锁。有了这种严格的顺序，就不会出现循环等待（circular wait），也就不会有死锁。

当然，在更复杂的系统中，锁会远不止两把，因此可能很难让所有锁形成全序（而且可能也没必要）。这时，**偏序**（partial ordering）就是安排获取锁的一种有效方式，可以用来避免死锁。Linux 的内存映射（memory mapping）代码 [T+94]（v5.2）就是偏序（partial ordering）加锁的一个绝佳的真实例子：源代码顶部的注释列出了十组不同的获取锁顺

提示：按锁的地址规定加锁顺序

有时候，一个函数必须获取两把（或更多）锁；我们知道，这时必须小心，否则可能发生死锁。设想有一个函数，调用方式如下：`do_something(mutex_t *m1, mutex_t *m2)`。如果代码总是先获取 `m1` 再获取 `m2`（或者总是先 `m2` 后 `m1`），就可能发生死锁，因为一个线程可能调用 `do_something(L1, L2)`，而另一个线程同时调用 `do_something(L2, L1)`。

为了避免这个问题，聪明的程序员可以用每把锁的地址来规定获取锁的顺序。只要按地址从高到低或者从低到高的顺序获取锁，无论传入的锁是什么顺序，`do_something()` 都能保证总以相同的顺序获取它们。代码大概是这样的：

```
if (m1 > m2) { // 按地址从高到低的顺序获取
  pthread_mutex_lock(m1);
  pthread_mutex_lock(m2);
} else {
  pthread_mutex_lock(m2);
  pthread_mutex_lock(m1);
}
// 代码假定 m1 != m2（不是同一把锁）
```

用这个小技巧，程序员就能简单高效地获取多把锁，而且不会死锁。

序，既有像“i_mutex 先于 i_mmap_rwsem”这样简单的，也有像“i_mmap_rwsem 先于 private_lock 先于 swap_lock 先于 i_pages lock”这样更复杂的。

可以想象，无论全序（total ordering）还是偏序（partial ordering），都需要精心设计加锁策略（locking strategy），构建时必须格外小心。此外，顺序只是一种约定，粗心的程序员很容易无视加锁协议（locking protocol），从而可能导致死锁。最后，规定加锁顺序（lock ordering）需要深入理解代码库，弄清各个例程之间的调用关系；只要犯一个错误，就可能招来那个以“D”开头的词[1]。

持有并等待

要消除死锁所需的持有并等待（hold-and-wait）条件，可以一次性、原子地（atomically）获取所有锁。实践中可以这样实现：

```
1 pthread_mutex_lock(prevention); // 开始获取
2 pthread_mutex_lock(L1);
3 pthread_mutex_lock(L2);
4 ...
5 pthread_mutex_unlock(prevention); // 结束
```

这段代码先获取锁 `prevention`，保证了获取锁的过程中不会发生不合时宜的线程切换，于是又一次避免了死锁。当然，这要求任何线程在获取任何锁时，都要先获取全局的

[1]提示：“D”指的是“Deadlock”（死锁）。

prevention 锁。例如，如果另一个线程试图按不同的顺序获取锁 L1 和 L2，也不会有问题，因为它这样做时持有 prevention 锁。

注意，这个方案有不少问题。和前面一样，封装又在跟我们作对：调用一个例程时，这种方法要求我们确切地知道必须持有哪些锁，并提前获取它们。这种技术还可能降低并发性（concurrency），因为所有锁都必须提前（一次性）获取，而不是在真正需要的时候才获取。

不可抢占

我们一般认为，锁一旦拿到，就会一直持有到调用解锁（unlock）为止。获取多把锁之所以常常惹麻烦，是因为在等一把锁的时候，我们手里还拿着另一把。许多线程库（thread library）提供了一组更灵活的接口来帮助避免这种情况。具体来说，例程 pthread_mutex_trylock() 要么在锁可用时获取锁并返回成功，要么返回一个错误码，表示锁已被别人持有；在后一种情况下，如果你还想获取这把锁，可以稍后再试。

借助这个接口，可以像下面这样构建一个不会死锁、而且不受加锁顺序（lock ordering）影响的获取锁协议：

```
1 top:
2   pthread_mutex_lock(L1);
3   if (pthread_mutex_trylock(L2) != 0) {
4       pthread_mutex_unlock(L1);
5       goto top;
6   }
```

注意，另一个线程可以遵循同样的协议，但按另一种顺序（先 L2 后 L1）获取锁，程序仍然不会死锁。不过，这确实带来了一个新问题：**活锁**（livelock）。两个线程可能（虽然概率不大）都在反复尝试这个序列，却总是没法同时拿到两把锁。在这种情况下，双方都在一遍又一遍地运行这段代码，所以这不是死锁，却没有任何进展，“活锁（livelock）”这个名字就是这么来的。活锁（livelock）问题也有解决办法：例如，可以在跳回去重新尝试整个过程之前，加一段随机延迟，从而降低竞争线程之间反复相互干扰的概率。

关于这个方案还要补充一点：它略过了 trylock 方法真正难办的地方。第一个难点仍然来自封装：如果某把锁深埋在被调用的某个例程里，跳回开头就更难实现了。如果代码在这个过程中已经获取了 L1 以外的一些资源，就必须确保把它们也小心地释放掉；例如，如果代码在获取 L1 之后分配了一些内存，那么在获取 L2 失败时，就必须先释放这些内存，再跳回开头重试（retry）整个序列。不过，在有限的场合下（例如前面提到的 Java vector 方法），这种方法可以很好地工作。

你可能还注意到，这种方法并没有真正引入抢占（即从拥有锁的线程那里强行夺走锁），而是利用 trylock 方法，让开发者能够优雅地放弃锁的所有权（也就是抢占自己的所有权）。不过这是一种实用的方法，所以尽管它在这方面并不完美，我们还是把它放在这里。

互斥

最后一种预防技术是干脆不需要互斥。一般来说，我们知道这很难做到，因为要运行的代码确实有临界区（critical section）。那么我们能做什么呢？

Herlihy 提出，可以设计出完全不用锁的各种数据结构 [H91, H93]。这些**无锁**（lock-free）方法和与之相关的**无等待**（wait-free）方法，背后的思想很简单：利用强大的硬件指令，就能构建出不需要显式加锁的数据结构。

举个简单的例子，假设我们有一条比较并交换（compare-and-swap）指令。你可能还记得，这是硬件提供的一条原子指令（atomic instruction），它做的事情如下：

```
1 int CompareAndSwap(int *address, int expected, int new) {
2     if (*address == expected) {
3         *address = new;
4         return 1; // 成功
5     }
6     return 0; // 失败
7 }
```

现在，假设我们想用比较并交换（compare-and-swap），把某个值原子地增加一定的量。可以用下面这个简单的函数做到：

```
1 void AtomicIncrement(int *value, int amount) {
2   do {
3     int old = *value;
4   } while (CompareAndSwap(value, old, old + amount) == 0);
5 }
```

这里没有先获取锁、更新、再释放锁，而是换了一种做法：用比较并交换（compare-and-swap）反复尝试把值更新为新值。这样就不需要获取锁，也就不会发生死锁；不过仍有可能发生活锁（livelock），因此健壮的解决方案会比上面这段简单代码复杂。

我们再看一个稍微复杂一点的例子：链表（linked list）插入。下面是在链表头部插入节点的代码：

```
1 void insert(int value) {
2   node_t *n = malloc(sizeof(node_t));
3   assert(n != NULL);
4   n->value = value;
5   n->next  = head;
6   head     = n;
7 }
```

这段代码执行的是简单的插入操作，但如果多个线程“同时”调用它，就会出现竞态条件（race condition，也叫竞争条件）。你能想出原因吗？（画一画：如果两个插入操作并发执行，链表会变成什么样？像往常一样，假设调度交错是恶意的。）当然，我们可以在这段代码前后获取和释放锁来解决这个问题：

```
1 void insert(int value) {
2   node_t *n = malloc(sizeof(node_t));
3   assert(n != NULL);
4   n->value = value;
5   pthread_mutex_lock(listlock);    // 临界区开始
6   n->next  = head;
7   head     = n;
```

```
8     pthread_mutex_unlock(listlock); // 临界区结束
9   }
```

在这个解决方案中，我们按传统的做法使用锁[2]。现在，我们换个做法，试试只用比较并交换（compare-and-swap）指令，用无锁（lock-free）的办法完成插入。下面是一种可能的写法：

```
1  void insert(int value) {
2    node_t *n = malloc(sizeof(node_t));
3    assert(n != NULL);
4    n->value = value;
5    do {
6      n->next = head;
7    } while (CompareAndSwap(&head, n->next, n) == 0);
8  }
```

这段代码先把 next 指针更新为指向当前的头节点，然后尝试把新创建的节点交换到链表头的位置。然而，如果在此期间有其他线程已经成功换入（swap in）了新的头节点，这次交换就会失败，于是这个线程会拿新的头节点再试一次。

当然，有用的链表光有插入操作是不够的；毫不奇怪，要构建一个支持插入、删除和查找的无锁（lock-free）链表，绝非易事。想了解更多，请阅读关于无锁（lock-free）和无等待（wait-free）同步的大量文献 [H01, H91, H93]。

通过调度避免死锁

在某些场景下，比起预防死锁，**避免**（avoidance）死锁更合适。避免死锁需要掌握一些全局信息，知道各个线程在执行过程中可能会获取哪些锁，然后据此调度这些线程，保证不会发生死锁。

例如，假设有两个处理器，要在上面调度四个线程。再假设我们知道：线程 1（T1）会在执行过程中的某个时刻、按某种顺序获取锁 L1 和 L2，T2 也会获取 L1 和 L2，T3 只获取 L2，而 T4 根本不获取锁。我们可以用表格列出这些线程的获取锁需求：

	T1	T2	T3	T4
L1	是	是	否	否
L2	是	是	是	否

于是，聪明的调度程序（scheduler）可以算出：只要 T1 和 T2 不同时运行，就绝不会发生死锁。下面就是一种这样的调度：



注意，T3 和 T1 重叠运行、或者 T3 和 T2 重叠运行，都没有问题。虽然 T3 会获取锁 L2，但它只获取一把锁，所以和其他线程并发运行时，永远不会导致死锁。

[2]细心的读者可能会问：为什么这么晚才获取锁，而不是一进入 `insert()` 就获取？细心的读者，你能想明白为什么这样做很可能是正确的吗？比如说，这段代码对 `malloc()` 的调用做了什么假设？

提示：不必事事做到完美（Tom West 定律）

Tom West 是经典计算机行业著作《新机器的灵魂》（Soul of a New Machine）[K81] 的主人公，他有一句名言："并非所有值得做的事都值得做好"，这是一条绝妙的工程格言。如果一件坏事很少发生，当然不应该花很大力气去预防它，尤其是在这件坏事发生的代价很小的时候。反过来，如果你在造航天飞机，出问题的代价是航天飞机爆炸，那么，嗯，也许你应该无视这条建议。

有些读者反对说："这听起来像是在建议用平庸来解决问题！"也许他们是对的，对这类建议确实应该谨慎。不过从我们的经验看，在工程界，面对紧迫的截止期限和其他现实问题，人们总要决定系统的哪些方面要做好，哪些方面先放一放。难点在于知道什么时候该做什么，这种眼光只有靠经验、靠全身心投入手头的工作才能获得。

我们再看一个例子。这一次，对相同资源（还是锁 L1 和 L2）的争用（contention）更激烈，如下面的争用表所示：

	T1	T2	T3	T4
L1	是	是	是	否
L2	是	是	是	否

具体来说，线程 T1、T2 和 T3 在执行过程中的某个时刻，都需要获取 L1 和 L2 这两把锁。下面是一种能保证永远不会发生死锁的调度：



可以看到，静态调度导致了一种保守的做法：T1、T2 和 T3 全都在同一个处理器上运行，于是完成这些任务的总时间大大延长了。这些任务本来也许可以并发运行，但出于对死锁的担心，我们没有这样做，代价就是性能。

这类方法中一个著名的例子是 Dijkstra 的银行家算法（Banker's Algorithm）[D64]，文献中还描述过许多类似的方法。遗憾的是，它们只在非常有限的环境中有用，例如嵌入式系统，在那里你完全清楚要运行哪些任务、它们各自需要哪些锁。此外，正如上面第二个例子所示，这类方法会限制并发性。因此，通过调度来避免死锁并不是一种广泛使用的通用方案。

检测与恢复

最后一种通用策略是：允许偶尔发生死锁，一旦检测到死锁再采取行动。例如，如果一个操作系统一年死机一次，你只要重启它，然后高高兴兴（或者骂骂咧咧）地继续干活就行了。如果死锁很少发生，这种算不上解决办法的办法其实相当务实。

许多数据库系统都采用死锁检测与恢复（deadlock detection and recovery）技术。死锁检测器（deadlock detector）定期运行，构建资源分配图（resource graph），检查其中有没有环。一旦出现环（cycle），也就是发生了死锁，系统就需要重启。如果需要先对数据结构做更复杂的修复，可能还得有人介入，帮着把这个过程理顺。

关于数据库并发、死锁及相关问题的更多细节，可以参考其他资料 [B+87, K87]。读读这些著作，或者更好的办法是去上一门数据库课程，深入了解这个丰富而有趣的话题。

32.4 小结

在本章中，我们研究了并发程序中会出现的几类 bug。第一类是非死锁 bug，它们出奇地常见，但通常比较容易修复。它们包括原子性违反（atomicity violation，本应一起执行的一串指令没有一起执行）和顺序违反（order violation，两个线程之间需要的顺序没有得到保证）。

我们还简要讨论了死锁：它为什么会发生，以及可以怎么应对。这个问题和并发本身一样古老，关于它的论文已经有好几百篇。实践中最好的解决办法是谨慎行事：制定一个获取锁的顺序，从一开始就防止死锁发生。无等待（wait-free）方法也很有前景，一些无等待的数据结构如今已经进入常用的库和关键系统，包括 Linux。然而，这类方法缺乏通用性，开发新的无等待（wait-free）数据结构又很复杂，这很可能会限制这种方法的整体实用性。也许最好的解决办法是开发新的并发编程模型（concurrent programming model）：在 MapReduce（来自 Google）[GD02] 这样的系统中，程序员可以描述某些类型的并行计算（parallel computation），而完全不需要任何锁。锁从本质上说就是有问题的；也许除非万不得已，我们都应该尽量避免用锁。

参考文献

[B+87] "Concurrency Control and Recovery in Database Systems" by Philip A. Bernstein, Vassos Hadzilacos, Nathan Goodman. Addison-Wesley, 1987. *The classic text on concurrency in database management systems. As you can tell, understanding concurrency, deadlock, and other topics in the world of databases is a world unto itself. Study it and find out for yourself.*

[C+71] "System Deadlocks" by E.G. Coffman, M.J. Elphick, A. Shoshani. ACM Computing Surveys, 3:2, June 1971. *The classic paper outlining the conditions for deadlock and how you might go about dealing with it. There are certainly some earlier papers on this topic; see the references within this paper for details.*

[D64] "Een algorithme ter voorkoming van de dodelijke omarming" by Edsger Dijkstra. 1964. Available: http://www.cs.utexas.edu/users/EWD/ewd01xx/EWD108.PDF. *Indeed, not only did Dijkstra come up with a number of solutions to the deadlock problem, he was the first to note its existence, at least in written form. However, he called it the "deadly embrace", which (thankfully) did not catch on.*

[GD02] "MapReduce: Simplified Data Processing on Large Clusters" by Sanjay Ghemawhat, Jeff Dean. OSDI '04, San Francisco, CA, October 2004. *The MapReduce paper ushered in the era of large-scale data processing, and proposes a framework for performing such computations on clusters of generally unreliable machines.*

[H01] "A Pragmatic Implementation of Non-blocking Linked-lists" by Tim Harris. International Conference on Distributed Computing (DISC), 2001. *A relatively modern example of the difficulties of building something as simple as a concurrent linked list without locks.*

[H91] "Wait-free Synchronization" by Maurice Herlihy. ACM TOPLAS, 13:1, January 1991. *Herlihy's work pioneers the ideas behind wait-free approaches to writing concurrent programs. These approaches tend to be complex and hard, often more difficult than using locks correctly, probably limiting their success in the real world.*

[H93] "A Methodology for Implementing Highly Concurrent Data Objects" by Maurice Herlihy. ACM TOPLAS, 15:5, November 1993. *A nice overview of lock-free and wait-free structures. Both approaches eschew locks, but wait-free approaches are harder to realize, as they try to ensure that any operation on a concurrent structure will terminate in a finite number of steps (e.g., no unbounded looping).*

[J+08] "Deadlock Immunity: Enabling Systems To Defend Against Deadlocks" by Horatiu Jula, Daniel Tralamazza, Cristian Zamfir, George Candea. OSDI '08, San Diego, CA, December 2008. *An excellent recent paper on deadlocks and how to avoid getting caught in the same ones over and over again in a particular system.*

[K81] "Soul of a New Machine" by Tracy Kidder. Backbay Books, 2000 (reprint of 1980 version). *A must-read for any systems builder or engineer, detailing the early days of how a team inside Data General (DG), led by Tom West, worked to produce a "new machine." Kidder's other books are also excellent, including "Mountains beyond Mountains." Or maybe you don't agree with us, comma?*

[K87] "Deadlock Detection in Distributed Databases" by Edgar Knapp. ACM Computing Surveys, 19:4, December 1987. *An excellent overview of deadlock detection in distributed database systems. Also points to a number of other related works, and thus is a good place to start your reading.*

[L+08] "Learning from Mistakes — A Comprehensive Study on Real World Concurrency Bug Characteristics" by Shan Lu, Soyeon Park, Eunsoo Seo, Yuanyuan Zhou. ASPLOS '08, March 2008, Seattle, Washington. *The first in-depth study of concurrency bugs in real software, and the basis for this chapter. Look at Y.Y. Zhou's or Shan Lu's web pages for many more interesting papers on bugs.*

[T+94] "Linux File Memory Map Code" by Linus Torvalds and many others. Available online at: https://github.com/torvalds/linux/blob/master/mm/filemap.c. *Thanks to Michael Walfish (NYU) for pointing out this precious example. The real world, as you can see in this file, can be a bit more complex than the simple clarity found in textbooks…*

 © 2008–26, ARPACI-DUSSEAU

作业（编程）

这个作业让你研究一些真实的、会死锁或能避免死锁的代码。不同版本的代码，对应着在一个简化的 `vector_add()` 例程中避免死锁的不同方法。关于这些程序以及它们共用的基础代码，详见 README。

问题

1. 首先，研究 `vector-deadlock.c` 中的代码，以及 `main-common.c` 和相关文件，弄清这些程序大致是怎么工作的，以及有哪些关键选项。然后运行：

   ```
   ./vector-deadlock -n 2 -l 1 -v
   ```

   它会创建两个线程（`-n 2`），每个线程做一次向量加法（`-l 1`），并且以详细模式（`-v`）运行。确保你理解它的输出。每次运行，输出有什么变化？

2. 现在加上 `-d` 选项，并把循环次数（`-l`）从 1 改成更大的数。会发生什么？代码（总是）会死锁吗？

3. 改变线程数（`-n`）会如何影响程序的结果？有没有哪些 `-n` 的值能保证不发生死锁？

4. 现在看看 `vector-global-order.c` 中的代码。首先，确保你理解这段代码想做什么；你明白它为什么能避免死锁吗？另外，在这个 `vector_add()` 例程中，为什么源向量和目标向量相同时要做特殊处理？

5. 现在用以下选项运行代码：`-t -n 2 -l 100000 -d`。代码要多长时间才能完成？增加循环次数或线程数时，总时间如何变化？

6. 如果打开并行（parallelism）选项（`-p`），会发生什么？每个线程处理不同向量的相加（这就是 `-p` 的作用），和处理相同的向量相比，你预计性能会有多大变化？

7. 现在研究 `vector-try-wait.c`，先确保你理解这段代码。其中第一次调用 `pthread_mutex_trylock()` 真的有必要吗？然后运行这段代码。和全局顺序方法相比，它运行得有多快？随着线程数增加，代码统计的重试次数如何变化？

8. 现在看看 `vector-avoid-hold-and-wait.c`。这种方法的主要问题是什么？分别在加 `-p` 和不加 `-p` 的情况下运行，它的性能和其他版本相比如何？

9. 最后，我们来看看 `vector-nolock.c`。这个版本完全不使用锁；它提供的语义和其他版本完全一样吗？为什么？

10. 现在比较它和其他版本的性能，分别考虑线程处理同样两个向量（不加 `-p`）和每个线程处理各自不同的向量（加 `-p`）这两种情况。这个无锁（lock-free）版本的性能如何？

 © 2008–26, ARPACI-DUSSEAU